

How post-link optimization unlocks speed that compilers leave behind.

Harshitha C

IBM – Power System Performance

The Problem: Dispersed Hot Code

- Frequently executed code ("hot code") is scattered across millions of lines of binary, mixed with rarely used "cold code."

- This leads to critical performance penalties:

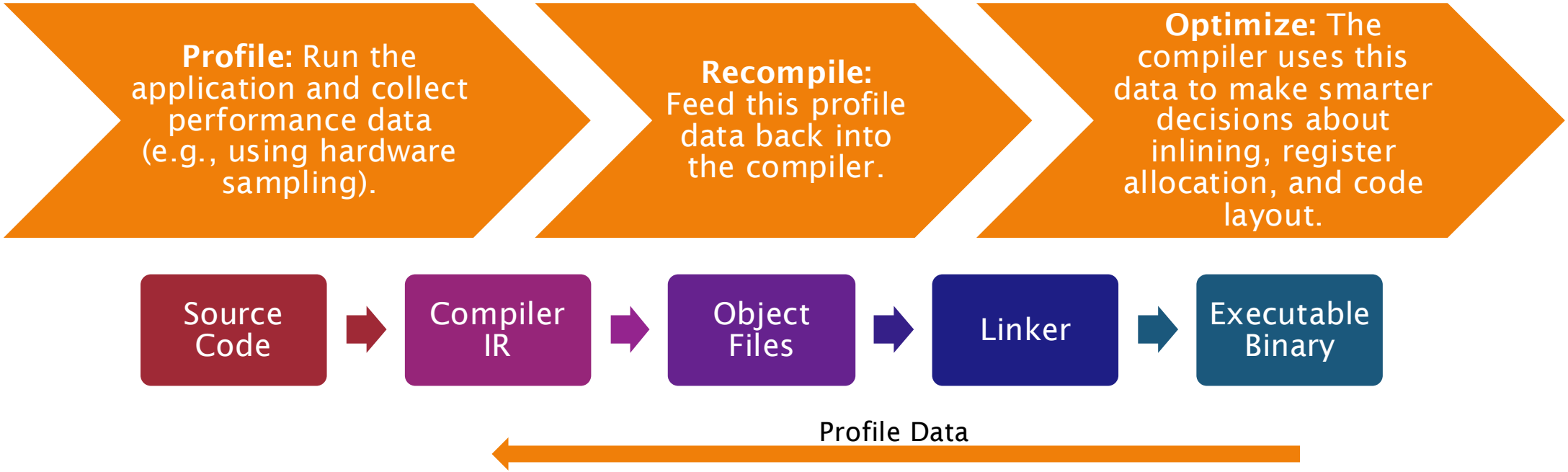
  - **Instruction Cache Misses:** The CPU constantly fetches new code instead of finding it in its fast, local cache.

  - **I-TLB Misses:** The CPU struggles to translate virtual to physical addresses for code spread across many memory pages.

  - **Branch Mispredictions:** The branch predictor is less effective when hot code paths are not laid out sequentially.

The Standard Solution: Let the Compiler Optimize with Profile Data

Feedback-Driven Optimization (FDO), or Profile-Guided Optimization (PGO), is the industry-standard approach.



In general, the earlier in the compilation chain the profile information is inserted, the larger the potential for its impact

Introducing BOLT: A Post-Link Optimizer That Sees the Final Picture.

What is BOLT?

- A modern, open-source binary optimizer built on the LLVM framework.
- It operates directly on the final compiled executable, after the compiler and linker are finished.
- It uses high-fidelity, sample-based profiling data to rewrite the binary for optimal code locality.

BOLT: A Post-Link Optimizer



Function Discovery



Read debug info



Read profile data



Disassembly



CFG construction Optimization



Pipeline Optimization

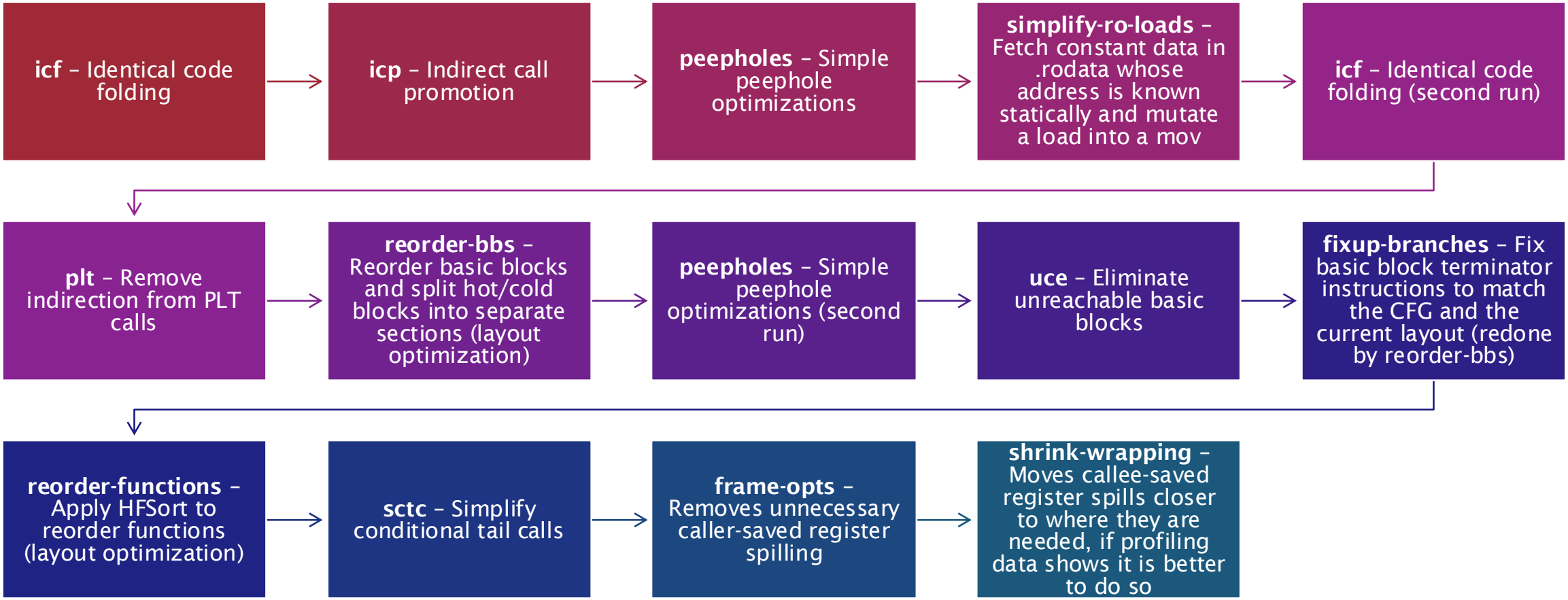


Emit and link functions

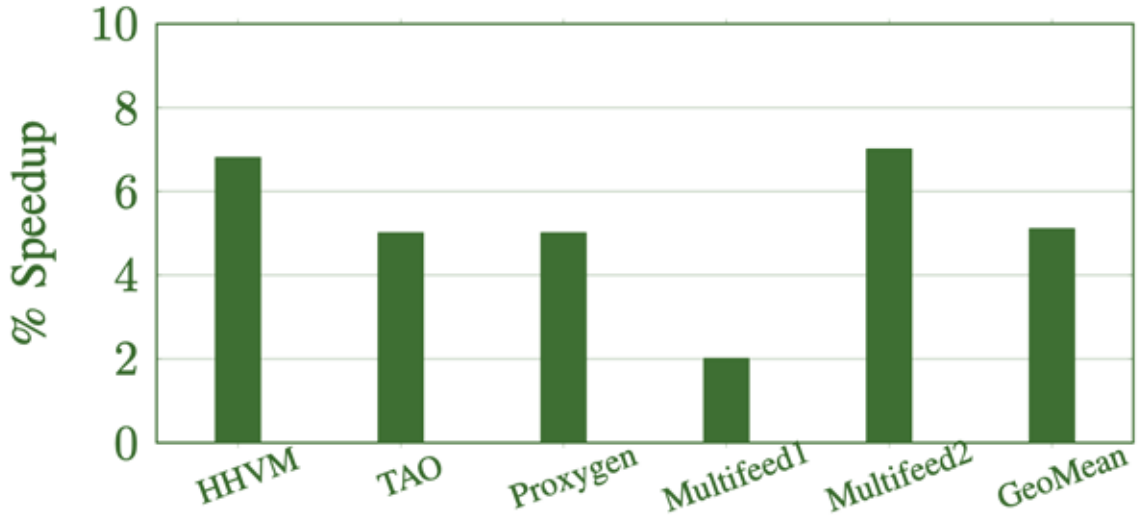


Rewrite binary file

BOLT: Optimization Steps



BOLT: Performance impact (Literature)



- For data-center applications, BOLT achieves up to 7.0% performance speedups on top of profile-guided function reordering and LTO.
- For the GCC and Clang compilers, studies shows that BOLT speeds up their binaries by up to 20.4% on top of FDO and LTO, and up to 52.1% if the binaries are built without FDO and LTO.

Impact of BOLT – Postgres Database PGBENCH workload

Core Count	Scaling Factor	TXN TYPE	Query Mode	#Clients	#Threads	RUNTIME(s)	us+sy	TXNS processed	Latency Avg (ms)	Initial Connection Time (ms)	TPS	AVG
10	100	TPC-B (sort of)	Simple	20	20	300	63	12031254	0.50	9.10	40102.69	
10	100	TPC-B (sort of)	Simple	20	20	300	63	12006795	0.50	7.66	40020.19	
10	100	TPC-B (sort of)	Simple	20	20	300	64	11945591	0.50	7.24	39818.87	39980.58

Core Count	Scaling Factor	TXN TYPE	Query Mode	#Clients	#Threads	RUNTIME(s)	us+sy	TXNS processed	Latency Avg (ms)	Initial Connection Time (ms)	TPS	AVG
10	100	TPC-B (sort of)	Simple	20	20	300	71	15711083	0.382	9.604	52371.71	
10	100	TPC-B (sort of)	Simple	20	20	300	71	15437663	0.389	9.437	51459.87	
10	100	TPC-B (sort of)	Simple	20	20	300	71	15530993	0.386	6.05	51769.98	51867.19

Calculated MAX Uplift: +7% over PGO

Impact of BOLT – Postgres Database PGBENCH workload

- Basic block reordering modified layout of 541 functions (~3.5% of overall functions)
- Splitting separates 266792 hot bytes from 593468 cold bytes
- 83 Functions were reordered by LoopInversionPass
- 18781 instructions were shortened
- BOLT-INFO: ICF folded 14 out of 14310 functions in 3 passes. 0 functions had jump tables.
- BOLT-INFO: Removing all identical functions will save 0.16 KB of code space. Folded functions were called 531607 times based on profile.
- BOLT-INFO: ICP Total indirect calls = 33268759, 155 callsites cover 99% of all indirect calls
- BOLT-INFO: ICP total indirect callsites with profile = 117
- BOLT-INFO: ICP total jump table callsites = 72
- BOLT-INFO: ICP total number of calls = 50916043
- BOLT-INFO: ICP percentage of calls that are indirect = 41.8%
- BOLT-INFO: ICP percentage of indirect calls that can be optimized = 84.4%
- BOLT-INFO: ICP percentage of indirect callsites that are optimized = 79.5%
- BOLT-INFO: ICP percentage of method calls candidates that have loads eliminated = 0.0%
- BOLT-INFO: ICP percentage of indirect branches that are optimized = 36.6%
- BOLT-INFO: ICP percentage of jump table callsites that are optimized = 50.0%

482473757	executed forward branches	6.00%
34115649	taken forward branches	-33.50%
57959010	executed backward branches	3.60%
29016116	taken backward branches	-20.50%
14432679	executed unconditional branches	-23.70%
154628277	all function calls	0.80%
13443407	indirect calls	-55.30%
9779819	PLT calls	14.30%
4574340062	executed instructions	1.00%
1066097932	executed load instructions	-0.30%
7321270	taken jump table branches	-36.70%
554865446	total branches	3.90%
77564444	taken branches	-27.30%
477301002	non-taken conditional branches	11.70%
63131765	taken conditional branches	-28.10%
540432767	all conditional branches	4.90%



THANK YOU ...

ANY QUERIES ??

Let's Connect: https://www.linkedin.com/in/harshitha-c1996/
harshu.harshitha5@gmail.com